Part01

1. What are the benefits of using a generic sorting algorithm over a non-generic one?

Code reuse for any data type

Type safety reduces errors

Flexible to pass different comparison methods

2. How do lambda expressions improve the readability and flexibility of sorting methods?

Lambdas make code shorter and easier to read

They allow inline dynamic definition of comparison logic

Improves flexibility by passing custom logic directly

3. Why is it important to use a dynamic comparer function when sorting objects of various data types?

Enables sorting different types without rewriting methods

Supports custom ordering rules

Increases code flexibility and reusability

4. How does implementing IComparable in derived classes enable custom sorting?

Allows derived objects to define their own comparison logic

Supports sorting by specific properties like Salary or Name

Works seamlessly with generic sorting methods

5. What is the advantage of using built-in delegates like Func<T, T, TResult> in generic programming?

Reduces boilerplate code

Enhances code reusability

Allows passing custom logic without creating new delegate types

6. How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

Lambdas are shorter clearer and more readable than full anonymous function syntax

Both are efficient but lambdas reduce verbosity and improve inline readability

7. Why is the use of generic methods beneficial when creating utility functions like Swap?

Works with any data type without rewriting code

Increases reusability and reduces duplication

Ensures type safety

8. What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

Challenges maintaining correct order handling tie-breakers

Benefits flexible sorting by multiple fields reusable across types

9. Why is the default(T) keyword crucial in generic programming, and how does it handle value and reference types differently?

Provides a safe default value for any type

Value types get zero-equivalent reference types get null

Ensures generic methods work correctly without knowing the type at compile-time

10. How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

Constraints enforce that T supports required operations

Prevents runtime errors

Makes code predictable and safe

11. What are the benefits of using delegates for string transformations in a functional programming style?

Allows passing different transformations easily

Promotes code reuse

Supports a clear modular functional style

12. How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

Allows defining operations separately and reusing them

Supports dynamic operation assignment

Reduces code duplication

13. What are the advantages of using generic delegates in transforming data structures?

Works for any input/output types

Increases reusability

Reduces boilerplate code for different types

14. How does Func simplify the creation and usage of delegates in C#?

Provides predefined delegate types

Reduces need for custom delegate declarations

Makes code cleaner and easier to read

15. Why is Action preferred for operations that do not return values?

Designed specifically for void-returning operations

Avoids unnecessary return type handling

Simplifies code for side-effect actions like printing

16. What role do predicates play in functional programming, and how do they enhance code clarity?

Encapsulate true/false conditions in reusable functions

Simplify filtering logic

Improve readability and maintainability

17. How do anonymous functions improve code modularity and customization?

Allow defining logic inline without naming functions

Makes code modular and adaptable

Supports quick custom behavior for specific cases

18. When should you prefer anonymous functions over named methods in implementing mathematical operations?

For short single-use operations

When readability and inline definition are preferred

Reduces code clutter

19. What makes lambda expressions an essential feature in modern C# programming?

Concise readable syntax

Inline definition of logic for collections delegates and LINQ

Enhances flexibility and maintainability

20. How do lambda expressions enhance the expressiveness of mathematical computations in C#?

Allow inline mathematical logic without separate methods

Make operations more readable and expressive

Facilitate functional programming and LINQ usage

Part02

2. Search about these topics (Parallel Programming and Concurrency - Unit Testing and Test-Driven Development (TDD) - Asynchronous Programming with async and await)

1. Concurrency & Parallel Programming:
Concurrency: Managing multiple tasks by switching between them (interleaving).
Parallelism: Executing multiple tasks simultaneously using multiple CPU cores.
Goal: Maximizing CPU efficiency and application performance.

2. Async & Await (Asynchronous Programming):
Async: A keyword used to define a method that performs a long-running task without blocking the main thread.
Await: A command that pauses the execution of a method until the task is complete, allowing the application to remain responsive.
Goal: Preventing UI freezing and improving user experience.

3. Unit Testing & TDD:
Unit Testing: Testing the smallest parts of an application (functions/methods) in isolation to ensure they work correctly.

TDD (Test-Driven Development): A workflow where you write the Test first, see it Fail, then write the Code to make it Pass.

1. LinkedIn article about Delegates in implementing functional paradigm

Mohamed Metwally • You
Software Engineer | Full Stack Web Developer
11h •

Delegates في C#...

لما تتعلم C#، هتقابل كلمة Delegate، وساعتها هتسأل نفسك: "طالما أقدر أنادي على أي function على طول، ليه أحتاج Delegate؟"

بص كده، لما تنادي على function مباشرة، الكود عارف بالظبط هينفذ إيه. لو عايز تغيّر حاجة، لازم تعدل الكود. الموضوع ثابت ومربوط بدالة واحدة.

الـ Delegate بيغير الفكرة دي. هو زي ما تقول ماسك "ريموت" للدالة. بدل ما تنفذها على طول، تمسكها كمرجع، وتبعتها لدالة تانية أو تختارها وقت ما تحب. كده الكود يبقى مرن وما يبقاش مربوط بحاجة واحدة.

تخيل عندك زرار في برنامج. أحيانًا يشغل نور، أحيانًا يشغل مروحة، أحيانًا يشغل تكييف. الزرار نفسه ثابت، لكن اللي وراه بيتغير. ده بالظبط فكرة Delegate.

كمان Delegate يخليك تبعت function جوه function بسهولة. بدل ما تكتب نفس الكود لكل حالة، تبعت الدالة المناسبة وتنفذها. كده الكود أنضف وأسهل في التعديل والصيانة.

لازم تعرف إن أي دالة تشتغل مع Delegate لازم يكون ليها نفس الشكل من حيث المدخلات والناتج. Delegate بيشتغل كعقد يضمن إن كل حاجة ماشية صح وما يحصلش غلط وقت التنفيذ

Show translation

Part03

1. **what is Asynchronous programming**
   Asynchronous programming is a way of writing programs so that tasks that take a long time (like reading a file, downloading data from the internet, or accessing a database) do not block the main program.

   Instead of waiting for a task to finish, the program can continue executing other code. Once the long-running task completes, it notifies the program and its result can be handled.